Имплементации на структури от данни

LinkedList (Свързан списък)

```
#include <iostream>
using namespace std;

class LinkedList {
 private:
  struct Node {
    int data;
    Node* next;
    Node(int d, Node* n) : data(d), next(n) {}
  };

  Node* first;
  Node* last;

  void copy(const Node* otherNode) {
    while (otherNode) {
      insertLast(otherNode->data);
      otherNode = otherNode->next;
    }
  }

  void free() {
    while (first) {
      Node* toDel = first;
      first = first->next;
      delete toDel;
    }

    first = last = nullptr;
  }

  void emptyInsert(int x) {
    Node* newNode = new Node(x, nullptr);
    first = last = newNode;
  }

  void removeSingleElem() {
    delete first;
    first = last = nullptr;
  }

 public:
  LinkedList() : first(nullptr), last(nullptr) {}

  LinkedList(const LinkedList& other) : first(nullptr), last(nullptr) {
    copy(other.first);
  }

  LinkedList& operator=(const LinkedList& other) {
    if (this != &other) {
      free();
      copy(other.first);
    }

    return *this;
  }
```

```
~LinkedList() { free(); }

// O(1)
void insertLast(int x) {
  if (isEmpty()) {
    emptyInsert(x);
    return;
  }

  Node* newLast = new Node(x, nullptr);
  last->next = newLast;
  last = newLast;
}

// O(1)
void insertFirst(int x) {
  if (isEmpty()) {
    emptyInsert(x);
    return;
  }

  Node* newFirst = new Node(x, first);
  first = newFirst;
}

// O(n)
void removeLast() {
  if (isEmpty()) {
    throw runtime_error("Empty list");
  }

  if (first == last) {
    removeSingleElem();
    return;
  }

  Node* newLast = first;
  while (newLast->next != last) {
    newLast = newLast->next;
  }

  newLast->next = nullptr;
  delete last;
  last = newLast;
}

// O(1)
void removeFirst() {
  if (isEmpty()) {
    throw runtime_error("Empty list");
  }

  if (first == last) {
    removeSingleElem();
    return;
  }

  Node* newFirst = first->next;
  delete first;
  first = newFirst;
}

// O(n)
bool contains(int x) const {
```

```
    Node* iter = first;
    while (iter) {
      if (iter->data == x) {
        return true;
      }
    }

    return false;
  }

  // O(1)
  bool isEmpty() const { return first == nullptr; }
};
```

Queue (Опашка)

```
class Queue {
 private:
  struct Node {
    int data;
    Node* next;
    Node(int d, Node* n) : data(d), next(n) {}
  };

  Node* first;
  Node* last;

  void copy(const Node* otherNode) {
    while (otherNode) {
      enqueue(otherNode->data);
      otherNode = otherNode->next;
    }
  }

  void free() {
    while (first) {
      Node* toDel = first;
      first = first->next;
      delete toDel;
    }

    first = last = nullptr;
  }

  void emptyInsert(int x) {
    Node* newNode = new Node(x, nullptr);
    first = last = newNode;
  }

 public:
  Queue() : first(nullptr), last(nullptr) {}

  Queue(const Queue& other) : first(nullptr), last(nullptr) {
    copy(other.first);
  }

  Queue& operator=(const Queue& other) {
    if (this != &other) {
      free();
      copy(other.first);
    }
```

```
    return *this;
  }

  ~Queue() { free(); }

  // O(1)
  void enqueue(int x) {
    if (isEmpty()) {
      emptyInsert(x);
      return;
    }

    Node* newNode = new Node(x, nullptr);
    last->next = newNode;
    last = newNode;
  }

  // O(1)
  int dequeue() {
    if (isEmpty()) {
      throw runtime_error("Empty queue");
    }

    int data = first->data;

    Node* toDel = first;
    first = first->next;
    delete toDel;

    return data;
  }

  // O(1)
  int peek() const {
    if (isEmpty()) {
      throw runtime_error("Empty queue");
    }

    return first->data;
  }

  // O(1)
  bool isEmpty() const { return first == nullptr; }
};
```

Stack (Стек)

```
class Stack {
 private:
  struct Node {
    int data;
    Node* next;
    Node(int d, Node* n) : data(d), next(n) {}
  };

  Node* first;

  void copy(const Node* otherNode) {
    while (otherNode) {
      push(otherNode->data);
```

```
      otherNode = otherNode->next;
    }
  }

  void free() {
    while (first) {
      Node* toDel = first;
      first = first->next;
      delete toDel;
    }

    first = nullptr;
  }

 public:
  Stack() : first(nullptr) {}

  Stack(const Stack& other) : first(nullptr) { copy(other.first); }

  Stack& operator=(const Stack& other) {
    if (this != &other) {
      free();
      copy(other.first);
    }

    return *this;
  }

  ~Stack() { free(); }

  // O(1)
  void push(int x) {
    Node* newNode = new Node(x, first);
    first = newNode;
  }

  // O(1)
  int pop() {
    if (isEmpty()) {
      throw runtime_error("Empty Stack");
    }

    int data = first->data;

    Node* toDel = first;
    first = first->next;
    delete toDel;

    return data;
  }

  // O(1)
  int top() const {
    if (isEmpty()) {
      throw runtime_error("Empty Stack");
    }

    return first->data;
  }

  // O(1)
  bool isEmpty() const { return first == nullptr; }
};
```

BinaryTree (Двоично дърво)

```
class BinaryTree {
 private:
  struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int d, Node* l, Node* r) : data(d), left(l), right(r) {}
  };

  Node* root;

  Node* copy(const Node* otherNode) {
    if (otherNode == nullptr) {
      return nullptr;
    }

    return new Node(otherNode->data, copy(otherNode->left),
                    copy(otherNode->right));
  }

  void free(Node* root) {
    if (root == nullptr) {
      return;
    }

    // delete left -> right -> root
    free(root->left);
    free(root->right);
    delete root;
  }

 public:
  BinaryTree() : root(nullptr) {}
  BinaryTree(Node* otherRoot) : root(copy(otherRoot)) {}

  BinaryTree(const BinaryTree& other) : root(copy(other.root)) {}

  BinaryTree& operator=(const BinaryTree& other) {
    if (this != &other) {
      free(root);
      root = copy(other.root);
    }

    return *this;
  }

  ~BinaryTree() { free(root); }

  // O(n) - копира цялото ляво поддърво
  BinaryTree getLeftSubtree() const {
    if (isEmpty()) {
      throw runtime_error("Empty tree");
    }

    return BinaryTree(root->left);
  }

  // O(n) - копира цялото дясно поддърво
  BinaryTree getRightSubtree() const {
```

```
    if (isEmpty()) {
      throw runtime_error("Empty tree");
    }

    return BinaryTree(root->right);
  }

  // O(1)
  int getRootData() const {
    if (isEmpty()) {
      throw runtime_error("Empty tree");
    }

    return root->data;
  }

  // O(1)
  bool isEmpty() const { return root == nullptr; }
};
```

BST (Binary Search Tree - Двоично дърво за търсене)

```
class BST {
 private:
  struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int d, Node* l, Node* r) : data(d), left(l), right(r) {}
  };

  Node* root;

  Node* copy(const Node* otherNode) {
    if (otherNode == nullptr) {
      return nullptr;
    }

    return new Node(otherNode->data, copy(otherNode->left),
                    copy(otherNode->right));
  }

  void free(Node* root) {
    if (root == nullptr) {
      return;
    }

    // delete left -> right -> root
    free(root->left);
    free(root->right);
    delete root;
  }

  Node* insert(Node* root, int data) {
    // tree is empty
    if (root == nullptr) {
      return new Node(data, nullptr, nullptr);
    }

    // tree has at least 1 node
    if (data < root->data) {
```

```
    root->left = insert(root->left, data);
  } else if (data > root->data) {
    root->right = insert(root->right, data);
  }

  return root;
}

bool contains(const Node* root, int data) const {
  if (root == nullptr) {
    return false;
  }

  if (root->data == data) {
    return true;
  }

  if (data < root->data) {
    return contains(root->left, data);
  }

  return contains(root->right, data);
}

Node* findMin(Node* root) const {
  if (root == nullptr) {
    return nullptr;
  }

  if (root->left == nullptr) {
    return root;
  }

  return findMin(root->left);
}

Node* remove(Node* root, int data) {
  if (root == nullptr) {
    return nullptr;
  }

  if (data < root->data) {
    root->left = remove(root->left, data);
  } else if (data > root->data) {
    root->right = remove(root->right, data);
  } else {
    // 3 cases for removing the node
    if (root->left == nullptr) {
      Node* tmp = root->right;
      delete root;
      return tmp;
    }

    if (root->right == nullptr) {
      Node* tmp = root->left;
      delete root;
      return tmp;
    }

    Node* succ = findMin(root->right);
    root->data = succ->data;
    root->right = remove(root->right, succ->data);
  }
```

```
    return root;
  }

 public:
  BST() : root(nullptr) {}

  BST(const BST& other) : root(copy(other.root)) {}

  BST& operator=(const BST& other) {
    if (this != &other) {
      free(root);
      root = copy(other.root);
    }

    return *this;
  }

  ~BST() { free(root); }

  // O(log n) среден случай, O(n) най-лош случай
  void insert(int data) { root = insert(root, data); }

  // O(log n) среден случай, O(n) най-лош случай
  bool contains(int data) const { return contains(root, data); }

  // O(log n) среден случай, O(n) най-лош случай
  void remove(int data) { root = remove(root, data); }

  // O(1)
  bool isEmpty() const { return root == nullptr; }
};
```